

AgentOps 2.0

Autonomous IoT Fleet Intelligence — Complete Build & Deployment Journey

Live: agentops-050e.onrender.com

Repo: github.com/tjmhmddaarif/agentops

Built October 6, 2026 — concept to production in one day

58/58 tests

CI green

Production live

59 FPS

1. What the product is

An autonomous AIOps platform: a simulated IoT fleet streams telemetry; a hybrid anomaly-detection engine finds faults; an autonomous agent investigates, decides, remediates, *verifies recovery*, and closes or escalates incidents — every step persisted and streamed live to a cinematic dark dashboard wrapped in a WebGL particle-globe landing experience.

```
SIMULATED FLEET (12 devices × 7 metrics, 12 failure scenarios)
→ HYBRID DETECTION (robust z-score + fast/slow EWMA + rate-of-change + Isolation Forest)
→ INCIDENT MANAGER (persistence · cooldowns · correlated grouping · watchdog)
→ AUTONOMOUS AGENT (investigate → decide → remediate → verify → resolve/escalate)
→ SSE EVENT STREAM → REACT DASHBOARD (live)
→ PARTICLE GLOBE (30k real land points → dissolve → login → deep-space app background)
```

No LLM key required — deterministic rule engine by default; optional Anthropic mode with validated structured output and automatic rule fallback. Single Docker container serves API + UI. Login-gated. CI-gated. One-click rollback.

2. Architecture

```
React/Vite (dashboard, lazy three.js chunk)
  | REST (Pydantic schemas) + SSE (/api/events/stream, cookie-auth)
  ▼
FastAPI — SimulationEngine (tick-driven, seeded RNG)
```

- └ HybridDetector (zscore/ewma/rate/iforest → fused score)
- └ IncidentManager (lifecycle state machine)
- └ AgentRuntime (per-incident state machines, 12-tool registry)
- └ AuthService (PBKDF2, DB sessions, httpOnly cookies)
- └ SQLite via repository layer (batch writes, indexed)

Deliberately lightweight: no Kafka, Redis, or Celery. `DATABASE_URL` can move to PostgreSQL later with zero code changes.

3. Build phases (in order)

1. **Backend core** — env config, event bus, logging, SQLAlchemy models + repository, Pydantic schemas.
2. **Simulation** — seeded per-device baselines + diurnal drift, 12 failure scenarios, ambient injector, demo script.
3. **Detection** — robust z-score, EWMA divergence, gated ROC, per-device Isolation Forests; fusion + suppression.
4. **Incident manager** — lifecycle state machine, cross-metric grouping, offline watchdog.
5. **Tools + agent** — 12-tool registry (risk classes, idempotency), agent state machine, rule chains, recovery verification, LLM fallback.
6. **API + SSE** — full REST surface, simulation control, injection, analytics, one event stream.
7. **Tests** — 58 tests including a deterministic end-to-end autonomous-loop test.
8. **Frontend** — fleet grid, live charts, incident timeline drawer, analytics, simulation control, settings.
9. **Scenario validation** — all 12 failure scenarios verified end-to-end, with and without Isolation Forest.
10. **Auth** — PBKDF2 users, DB sessions, cookie middleware, landing/login, change-password.
11. **Cinematic globe** — Natural Earth → 30k particles, GPU dissolution, drag physics, starfield/dust/nebula.
12. **Ship** — Docker, compose, render.yaml, CI, docs → GitHub → Render production.

4. Real bugs found & fixed

#	BUG	ROOT CAUSE	FIX
1	Z-score stopped firing mid-fault	Window absorbed the anomaly into its baseline	Median/MAD robust statistics
2	EWMA never caught slow ramps	It chased the drift	Fast/slow divergence + gated adaptation
3	EWMA self-masked anomalies	Fault inflated residual variance	Freeze variance in abnormal regime + noise floor
4	Isolation Forest scoring 28 ms/call	sklearn joblib dispatch per call	Manual path-length scoring, validated vs sklearn (~60× faster)
5	IF scores compressed	Inliers sit near decision boundary	Sigmoid calibrated on training-score mean/std
6	Warmup false positives	Unstable early MAD estimates	Min-sample + override maturity gates
7	Recovery looked anomalous	Rolling window contaminated by incident	Verify against gated EWMA baseline
8	Restart didn't revive offline device	Offline ticks kept old maximum	restart() sets exactly 2 reboot ticks
9	Correlated failure → 3 parallel incidents	Grouping was per-metric only	Device-level correlated grouping
10	SSE test hung forever	TestClient can't consume infinite streams	Test SSE via real uvicorn thread
11	/incidents 404 on refresh	StaticFiles isn't an SPA	Catch-all fallback → index.html
12	Black screen after navigation	framer-motion exit phase stranded opacity 0	Keyed remount, no exit animation
13	Feed flooded by repeat anomalies	Detector emits per tick during persistence	Coalesce repeats in the store
14	Globe was a white blob	Point size 10× too large	Perspective size constant tuned (140→26)
15	Render deploy failed at startup	Relative DATABASE_URL resolved against container CWD; directory missing	_ensure_sqlite_parent_dir() + mkdir in Dockerfile; CI proved the fix
16	"Invalid credentials" on production	Expected: Render auto-generated ADMIN_PASSWORD	Read from Environment tab (by design)

5. Verification record

- **pytest: 58/58 passed** — simulator, detectors (incl. IF fast-path parity), agent rules/chains, idempotency, LLM fallback, API, auth, live SSE, deterministic end-to-end.
- **GitHub CI green on main** — Backend tests ✓ Frontend build ✓ Docker build + container health smoke ✓.
- **Browser-verified (Playwright)** — globe form/drag/dissolve, login, dashboard-over-space, incident drawer, analytics, simulation. **59 FPS / 27 MB heap**.
- **Production-verified** — health endpoint healthy, simulation running, API 401 without session, demo credentials hidden.

6. Commands

```
# Run locally (Windows)
./start-backend.ps1                # http://localhost:8000
cd frontend && npm run dev          # optional hot-reload UI on :5173

# Tests
cd backend && .venv/Scripts/python -m pytest tests -q

# Frontend build
cd frontend && npm run build

# Docker
docker compose up --build

# Daily dev loop (feature branches only — main is protected)
git checkout -b feature/<name>
# ... code ... tests + build must be green ...
git push -u origin feature/<name>   # → PR → CI → squash merge → Render auto-deploys
```

7. Production operations

TASK	WHERE
URL	https://agentops-050e.onrender.com
Admin password	Render → agentops → Environment → ADMIN_PASSWORD (auto-generated)
Change password	In-app Settings → Change Password
Redeploy	Automatic on git push to main (or Manual Deploy)
Rollback	Render → Events → previous deploy · flip FEATURE_FLAGS · git revert
Branch protection	main requires Backend tests + Frontend build + Docker build; force-push blocked
Free-tier nap	Sleeps after 15 min idle; ~50 s cold start — normal
Data	SQLite resets on redeploy/restart (demo re-seeds itself); PostgreSQL later via DATABASE_URL

8. Disaster recovery — restore on a new machine

```
# 1. Install: Python 3.12+, Node 20+, git
git clone https://github.com/tjmhmddaarif/agentops.git
cd agentops

# 2. Backend
python -m venv backend/.venv
backend/.venv/Scripts/pip install -r backend/requirements.txt # Windows
# backend/.venv/bin/pip install -r backend/requirements.txt # macOS/Linux

# 3. Frontend
cd frontend && npm install && npm run build && cd ..

# 4. Run
./start-backend.ps1
# open http://localhost:8000 — fleet boots; dev login: admin / agentops

# 5. Re-deploy anywhere: repo includes Dockerfile + render.yaml —
#   Render Blueprint or Railway picks them up automatically.
```

Secrets policy: the production admin password lives only in Render's Environment — never in git. Local dev credentials (admin/agentops) apply only to fresh local databases.

9. Feature flags — the experiment safety valve

```
FEATURE_FLAGS="predictive_health=true,chaos_controls=false"
```

Default OFF in main. Flag OFF = the feature 404s and its UI hides. Flip flags in Render → Environment for a sub-minute rollback without redeploying. Reference implementation: the `predictive_health` flag and its experimental failure-risk card.

10. Where to continue

- Alert webhooks on escalation; human-approval queue for HIGH_RISK tools
- Seasonal (hour-of-day) baselines and per-device adaptive thresholds
- Cross-device root-cause correlation; PostgreSQL persistence
- Custom domain / named Cloudflare tunnel

Process: feature branch → flag → CI green → squash merge → tag → auto-deploy. Rollback: flag flip (<1 min) → Render previous deploy (~2 min) → git revert (permanent).

Appendix — links

- Production: <https://agentops-050e.onrender.com>
- Repository: <https://github.com/tjmhmddaarif/agentops>
- Legacy (original Streamlit, archived): <https://github.com/tjmhmddaarif/agentops-legacy>
- CI: <https://github.com/tjmhmddaarif/agentops/actions>

Generated from docs/PROJECT_JOURNEY.md — keep both with the project.